Changes to old Class diagramm

- Generate subject mark is moved internally to Subject, since its literally just one line of code and doesnt have to be unique unlike the other generated ID'S
- Same with verifySubjectCount and verifyMark
    - Mark is just created as int between 25 & 100 so its always correct
    - Subject count is simply verified internally in the student class
    - **Core Logic of Classes belongs inside classes → Encapsulation**
- getPassFailStatus() as a method of Subject class is unnecessary, since it is only used for display purposes. The sample provided in the task description shows that marks are loaded anyways for pass/fail sorting, so the categorization could just be implemented in the Admin Controller
- Instead of returning Booleans for the status if operation was successful, we should raise Errors and handle them appropriately
- The Validator and Generator Classes should get the StudentRepository as an attribute during initialisation in order to use the provided **getAllStudentIDs() & getAllSubjectIDs()** functions to check wether ID's are unique

My Interfaces for you → StudentRepository

Initialization

StudentRepository shouldn't be initialized by using init but by using the createStudentRepository(database: Database) -> StudentRepository function, which automaticaly loads the students list from the databasea

Database should be created beforehand, probably with the the path to the students.data file as argument datbase = Database("data/students.data")

- **createStudentRepository(database: Database) -> StudentRepository**

Get Functions

All of your classes should interact with the functions of the StudentRepository Class.

Direkt access to Student or Subject Objects is not intended.

The get Functions return a **copy** of the requested Object.

- **getStudents() → list[Student]**
- **getStudent(student_id) → Student**

Set Functions

Set functions update the database everytime they are called

Since every change to the student information needs to be saved, the **provided set functions must be used** for any change to the data

This also counts for changes to subjects, e.g. to remove a subject a specific student needs to be collected a copy using the get function, then removing the subject with my_student.removeSubject(subject_id) and then **updating the student with the updateStudent(student_id)** function

- **addStudent(Student)**
- **removeStudent(student_id)**
- **updateStudent(student)**
- **clearAllStudents()**

Utility Functions

Functions that return a specific List are implemented for ease of use, mainly for the **Validator** and **Generator** classes for checking if email (student name), student id or subject id already exist

- **getAllStudentIDs() → list[int]**
- **getAllSubjectIDs() → list[int]**
- **getAllEmails() → list[str]**

Working with the StudentRepo

1. Checking out copy of student
2. Changing student (remove / add subject, etc.)
3. Update StudentRepo

- Example - adding subject:

```
# get student
student = repo.getStudent(student_id)

# create subject with Subject class methods & validator
subject_id = generator.generateSubjectId()
mark = Subject.generateSubjectMark()
grade = Subject.calculateGrade(mark)
subject = Subject(subject_id=subject_id, mark=mark, grade=grade)

# add subject and update StudentRepo
student.addSubject(subject)
repo.updateStudent(student)
```

Student methods

The Students functions provide manipulation and viewing of subject as well as the option to change the subject

- **addSubject(subject)**
- **removeSubject(subject_id)**
- **getSubjects() → list[Subject]**
- **changePassword(new_password)**

Subject - static methods

- The subject class has 2 static methods that should be used when creating a subject in the enrollment process

- **generateSubjectMark() → int**
- **calculateGrade(mark) → string**

Interfaces i need from your classes

Database

- StudentRepository hat a Database as attribute
- when the StudentRepository is created, the students list is filled through the database like this students = database.load()
    - **load()** should return the complete list with all Students and their respective attributes: subject, name, id etc.
- Everytime a change is made to the student list or to the students in the list, the change is saved by the StudentRepository class with the function: self.database.save(self.students)
    - **save(list[Students])** should safe the list to the student.data file in a suitable way so that it can be loaded again
    - I think a json like structure would be suitable